



Analyzing Scrum Team Impediments Using NLP

Kaleemunnisa , Christelle Scharff^(✉) , Krishna Mohan Bathula ,
and Kaiyin Chen

Pace University, Seidenberg School of CSIS, One Pace Plaza, New York, NY 10038, USA
{klnu, cscharff, kbathula, kc25295n}@pace.edu

Abstract. In this research, we focus on the impediments encountered by students in capstone projects following the Scrum methodology. Scrum meeting notes were collected in a dataset to permit Scrum roles and instructors to monitor progress and issues. We identified 9 categories of impediments in this dataset: Android, Coding Skills, Debugging, External Factors, Firebase/Database, Git/GitHub, Teamwork, Time Management, and UI/UX Design. We developed a Large Language Model (LLM) to classify these impediments. Natural Language Processing (NLP) has the potential to support software engineering processes. The novelty of this research is that it attempts to identify impediments faced by students' Scrum teams with AI and support students and instructors. The relevance of the approach was discussed with subject matter experts (SME) of the industry. The proposed model is useful in both the academic and industry settings, to identify on-the-fly areas that need attention and, if fixed, would increase team productivity.

Keywords: Agile · Artificial Intelligence (AI) · Impediments · Machine Learning · Large Language Model (LLM) · Natural Language Processing (NLP) · Scrum · Software Engineering Education

1 Introduction

Scrum is an agile methodology that was developed in the nineties with the focus on delivering value to all stakeholders incrementally [29]. It has been used in different types of projects including software development. Scrum is one of the most extensively adopted agile methodologies, ranking second in popularity after Kanban [9]. While Scrum does not explicitly recommend engineering practices, it has been widely used paired with Extreme Programming [7]. It emphasizes specific ceremonies such as the daily standup Scrum meeting, orchestrated by the Scrum Guardian (also known as Scrum Master), where development team members, provide individual work updates in the form of answers to three standardized questions: What did I do since yesterday?; What will I do until tomorrow?; and What are my impediments so far?. Impediments are factors that “block developers in their creation of a valuable piece of software in a sprint (iteration) or that restrict the team in achieving its intrinsic level of progress” [29]. Impediments go from insufficient skills, issues with tooling, illness of team members, unavailability of the Product Owner (key Scrum role in charge of the requirements), and lack of

alignment with the product vision, to conflicts between developers and workload of the team members. The Scrum Guardian helps the team to adopt and practice Scrum. The role facilitates the Scrum meeting, Sprint Planning, and Sprint Demo/Review; keeps the team accountable to itself and its commitment; creates an environment where raising impediments is safe; and, as a problem solver, works in removing impediments by seeking support within and outside the team. Assistance in the removal of impediments is a strategic role of the Scrum Guardian to ensure delivering the software within constraints of time, scope, and budget. The process goes through the timely identification and address of impediments, permitting continuous improvement.

This study focuses on a post-mortem analysis of projects realized in a software engineering capstone course. We analyzed the 515 Scrum meeting notes of 32 teams over 4 semesters (Spring 2021 to Fall 2022). We studied the impediments encountered by teams of students using Scrum and elicited during Scrum meetings when answering the standard question: “What are my impediments so far?” Understanding impediments helps teams achieve better results and higher success in delivering software increments. In addition, it provides instructors with more visibility of the issues students are encountering to address them timely. Running capstone courses is a time-consuming activity and any support to relieve instructors is useful. With timely knowledge of impediments, instructors can provide tailored and additional help to teams. They can propose individualized feedback, targeted training, and access to resources (infrastructure, human, technical etc.). Team awareness will increase, leading to less frustration and more trust.

The research questions we addressed in this study are:

1. What are the impediments met by Scrum teams and how can we classify them?
2. What type of machine learning model would be most appropriate to classify impediments?

By analyzing Scrum meeting notes, we identified 9 categories of impediments: Android, Coding Skills, Debugging, External Factors, Firebase/Database, Git/GitHub, Teamwork, Time Management, and UI/UX Design. We thought that it would be interesting to classify team impediments and apply this classification on-the-fly to address issues quickly. To achieve this, we developed and evaluated a Large Language Model (LLM) based on GPT-3.5 Turbo [14, 26].

This study has an impact on instructors and students. In addition, outside of the scope of education, this study can be useful for the industry. In this context, we sought feedback from two subject matter experts (SME) of the industry who validated and shared their vision on the potential of the approach.

This paper is organized as follows. Section 2 provides useful background. Section 3 presents the educational context for this study. It outlines the software engineering course and how the semester-long project is organized in terms of process and supportive tools. Section 4 presents the process used to identify the impediments from the Scrum meeting notes and discusses the types of impediments encountered by the teams. It answers the first research question. Section 5 outlines our model experimentation and presents our final model to automatically classify impediments using NLP. This section answers the second research question. Section 6 explains how we validated and discussed our overall approach with subject matter experts. Section 7 concludes our work and outlines future work.

2 Background

In this section, we provide background information related to the use of agile and scrum in the classroom, as well as the impact of AI on software engineering.

2.1 Agile in the Classroom

Scrum and agile methodologies have been introduced in the classroom setting to permit students to work iteratively, develop soft skills, and provide them with industry relevant and transferable skills. Most of the targeted courses for introducing Scrum are related to computer science and software engineering, especially capstone courses [4, 15, 19]. Research on the impact of Scrum in the classroom has shown promising results with teams increasing their motivation and productivity [21, 23, 30]. In [15], students experimented with Scrum in software engineering, human-computer interaction, and game technology courses. Scrum permitted them to stay on track and complete more work than expected. In capstone software engineering courses, instructors are designing scenarios to provide students with real experiences with agile methodologies, mimicking the industry environment. They provide training, assign roles to students, involve professional Scrum Guardians, Product Owners, and Agile Coaches, and organize the academic calendar to accommodate agile ceremonies and sprints. Instructors organize Scrum meetings, dedicate closer attention to group dynamics, provide individualized material for self-learning, identify students' affinity with tasks, and distribute the students to appropriate tasks [30]. The complex role and responsibilities undertaken by the instructor in Scrum projects has been discussed extensively in the literature [2]; instructors are often involved in project management or Scrum Guardian roles on top of their course responsibilities. The transition to agile methodologies often focuses, first, on the introduction of the Scrum meeting as one of the pillars of agile. In addition, Scrum meetings appear as the most popular technique after the burndown chart, sprint planning, and user stories (requirements) and estimation. There have been discussions on the difficulties for students to adhere to Scrum, including missing Scrum meetings, not answering all three relevant Scrum questions, late sprint planning, not preparing the sprint review, omitting the value section in user stories, and not using the burndown chart to monitor progress [23, 24]. In terms of success, it was observed that high-performing and low-performing teams use about the same Scrum practices, but the high-performing teams use Scrum practices more thoroughly by taking advantage of the Scrum and Review meetings [18]. Several studies are proposing configurations where students are distributed between institutions and countries, emphasizing working across distance, culture, and time zones [6, 23, 24]. Noteworthy, there is a gap between the implementation of Scrum in the classroom and in the industry as the stakes are not the same and students are likely unable to do the Scrum daily.

2.2 AI and Software Engineering

Artificial Intelligence (AI) is impacting software development and the role of software developers. Advances were possible thanks to the wide availability of data and the increasing computer power offered to deal with large and complex neural networks.

AI techniques have been applied to software development, classifying the points of application into three categories: process, product, and runtime [12]. Natural Language Processing (NLP) [5] has a crucial role to play in supporting software development [5]. Recently, large code datasets have permitted the development of GitHub Copilot [13] and Microsoft DeepCoder [1]; they provide real-time code suggestions following specifications and input-output requirements. Such tools encourage developers to focus on more complex code rather than boilerplate or repetitive code patterns. The literature also shows different usage of NLP in projects following the Scrum process. NLP is used to discover related user stories and scope them into released [25], transform user stories into UML use cases [10], and detect privacy requirements from user stories [3].

3 Educational Context

In this section, we describe the targeted software engineering capstone course, types of mobile applications developed by students, processes followed by the teams, and tooling infrastructure.

3.1 Capstone Course

The target course is the capstone software engineering course taught in Spring and Fall 2021 and 2022. The course is taught synchronously online. It covers process, requirements, design, and testing. It integrates presentations by software engineering practitioners and a semester-long project. Students are divided into teams to collaborate on developing Android applications using Scrum and GitHub for code collaboration and versioning. Most applications use Firebase as a backend. Students in the course are new to Android, Git/GitHub and Scrum. The software projects d focused on the topics of United Nation Sustainable Development Goals (SDG). Examples of projects include apps related to reducing carbon footprint, monitoring water quality, recycling, and increasing companies' transparency. 32 teams and 108 students took this course in the 4 targeted semesters.

3.2 Process and Tooling

Teams used Scrum with practices of Extreme Programming (XP). After the requirement phase, they completed three sprints of two weeks. Students were provided with training on Scrum via lectures, talks by professional Scrum Guardians, and by playing an online version of the BallPoint Game [28]. Before sprint 1, students learned Android through Google Codelabs and Git/GitHub through tutorials and collaborative in-class exercises that illustrated conflict situations. Teams all used a pre-defined tooling set. Tools can be categorized into engineering, communication, and project management as shown on Table 1. GitHub was used as a code and documentation repository. Documentation included idea proposal, product/sprint backlogs, sprint planning, and Scrum meeting and Scrum demo/review notes. For this study, we focus on data collected from the Scrum meeting notes.

Table 1. Tools Used in the Software Development Projects

Type of Tools	Use
Engineering	Android Studio for mobile app development; Firebase for back-end real-time databases and authentication; Git/GitHub for code versioning and bug reports development; Figma or similar for wireframing; Photoshop or similar for art production; APIs
Communication	Slack or Discord for team and instructor/team communications; Zoom for instruction and synchronous Scrum and Spring Reviews
Project Management	Google calendar with notifications; GitHub for project documentation and code deliveries; Google Docs for project documentation; YouTube for software demos

4 Classifying Scrum Impediments

This section presents the impediments encountered by students in the Scrum projects realized in the software engineering course described previously. It highlights the creation of the dataset, the process we used to identify the categories, the categories themselves, and the findings. It answers the first research question: What are the impediments met by Scrum teams and how can we classify them?

4.1 Scrum Impediments Dataset

In this study, we did a post-mortem analysis of the Scrum meeting artifacts. We created a dataset from the Scrum meeting notes written by students and gathered on GitHub. The dataset comprises 515 impediment entries. It is used for classifying impediments into categories. It is multiclass, as some sentences can be classified into two categories. Each entry is composed of one or several sentences that capture the same types of impediments.

4.2 Scrum Impediments Categories

After creating the dataset, we did a corpus analysis. The dataset has a 1303-word vocabulary. The lexical diversity is 7.65, which represents a moderate range of words. The most common words in the corpus are project-specific words like “sprint1”, “sprint2”, and “sprint3”, technical terms like “Android,” “Firebase,” and “Git,” as well as coding-related words like “coding,” “debugging,” and “classes”, and team-related words such as “teamwork” and “member”. Overall, the most common words in the corpus show how technical and collaborative the project was, and the importance of skills and tools for coding. The word cloud on Fig. 1 permits to visualize word occurrences. It also determined initial categories of impediments.

An instructor and a student in the class read all the Scrum meeting notes to label them. We coded each entry of the dataset and identified 9 categories: Android, Coding Skills, Firebase/Database, Debugging, External, Git, Teamwork, Time, and UI. The dataset was labeled with these 9 category names. Examples of impediments with their categories are provided in Table 2.

The second highest-ranked impediment is “Time”, which refers to time management and balancing workload. “Coding Skills” are also ranked second; they refer to technical skills required for software development, including the use of APIs and difficulties with Android Java. The lowest-ranked impediment is “Teamwork”; this may mean that students were comfortable to work as teams and that teamwork was not a main concern.

In terms of the evolution of the impediments through sprints, the main impediments that students faced during sprint 1 were “Android”, “Coding Skills” and “Time”. In sprint 2, they were “Coding Skills”, “Firebase” and “Time”. In sprint 3, they were “External”, “Time” and “Coding Skills”. While Scrum is a time-boxed methodology, “Time” is a

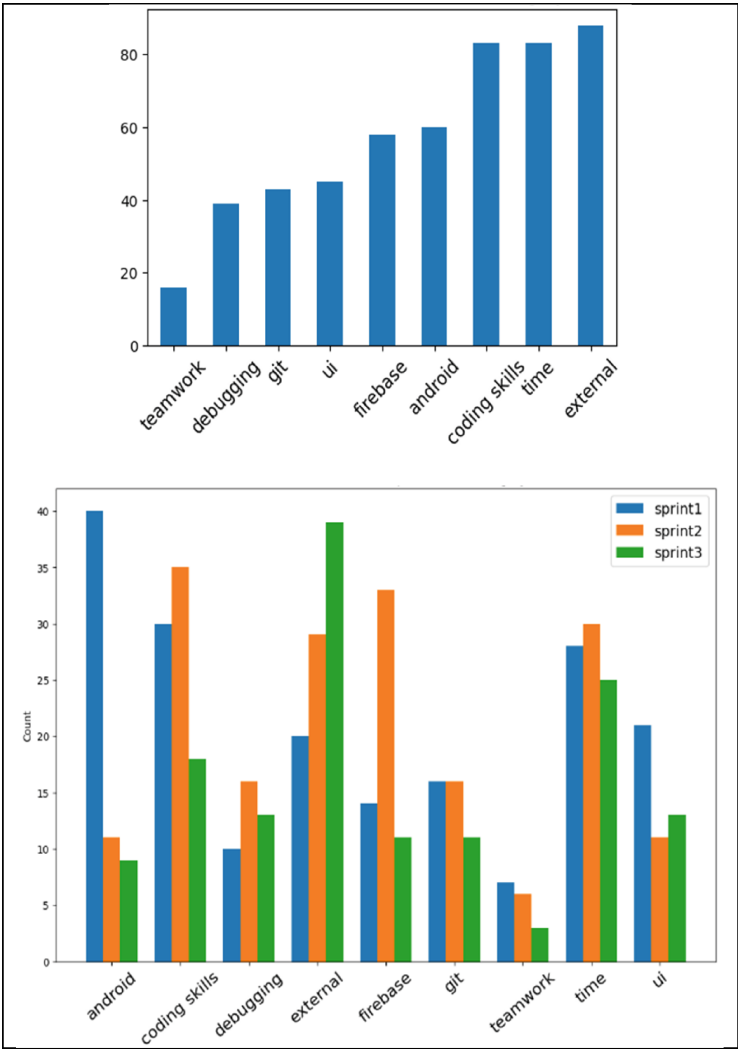


Fig. 2. Occurrences of Impediments Overall and in Each Sprint

constant impediment for students during sprints. Students are also facing “Coding Skills” impediments throughout the project. Students are still learning Android in sprint 1, are focusing on Firebase in Sprint 2, and are exposed to more “External” impediments in sprint 3, probably due to stress at the end of the semester.

5 Automated Classification of Impediments with NLP

This work is a first attempt to build a classification model based on NLP to classify impediments. In this section, we present the overall classification process and the model based on GPT-3.5 Turbo that we implemented and evaluated. This section answers the second research question: What type of machine learning model would be most appropriate to classify impediments?

5.1 Overall Classification Process

Figure 3 illustrates the overall classification process. It illustrates what input we provided to the classifier and the output we obtained.

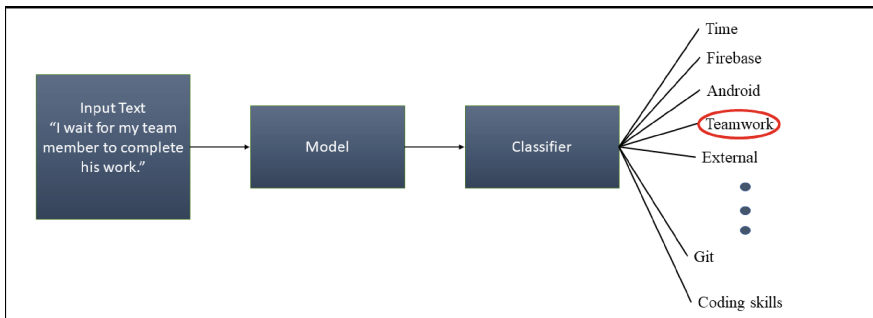


Fig. 3. Classification of Impediments

5.2 Building a LLM

Models for Text Classification. Machine learning algorithms can perform text classification tasks on labeled data. Multinomial Logistic Regression [27], Support Vector Machine (SVM), and K-Nearest Neighbors (KNN) [17] are popular for text classification. Deep Learning algorithms such as Convolutional Neural networks (CNN), known for image classification, can also be used on text classification with satisfactory results [32]. However, these techniques are not capable of comprehending the semantics of text. When working on small datasets, they require using data augmentation (e.g., random insertion, deletion, swapping). Bidirectional Encoder Representations from Transformers (BERT) models are based on the transformer architecture and pre-trained with a huge corpus of unlabeled data to permit increased capacity for understanding context

and ambiguity in language [8, 31]. Large Language Models (LLM) employ deep learning, are trained on huge amount of text, and have massive number of parameters from millions to billions, resulting in complex model architectures [26]. LLMs can generate text that closely resembles human language and perform diverse natural language processing tasks from language generation to translations and classification.

Choice of LLM. We experimented with Multinomial Logistic Regression, Multiclass SVM, KNN, CNN and BERT models to obtain a baseline, but, due to the small size of our dataset, only a LLM-based model permitted us to obtain promising results. For reasons of space and scope, we only present the LLM-based model in this paper. We used transfer learning and developed a classifier using the state-of-the-art Open AI GPT 3.5 Turbo LLM model [14]. GPT-3.5 Turbo is a neural network-based language model that was released in 2021 with 154 million parameters and trained on over 570 GB of data. It has less parameters than GPT-3 and it provides a balance between performance and cost-effectiveness during deployment. The implementation was done using the Scikit-LLM library [16]. The choice of the classifier was Dynamic Few-Shot Text Classification, which dynamically adds several examples to the input to contextualize and offer directions to the model. This is important as LLMs have limited context length and text inputs that need to be classified are of a wide range of sizes. For example, GPT-3.5 has a context limit of 4096 tokens.

Results. We used different metrics to evaluate the model. Accuracy, Precision, Recall, and F1 are the commonly used metrics used for classification. Accuracy checks if the data are properly classified. It is determined by dividing the number of correctly predicted labels as positive, also known as true positive (TP), and correctly predicted labels as negative, also known as true negative (TN), by the total number of examples of data $((TP + TN)/(TP + TN + FP + FN))$, where FP and FN refer to false positive and false negative respectively. Precision is the ratio of true positive predictions (positive examples that were correctly classified) by all positive predictions $(TP/(TP + FP))$. Recall is the ratio of correctly predicted labels divided by the correctly predicted labels and incorrectly predicted labels $(TP/(TP + FN))$. F1 is interpreted as a weighted harmonic mean of precision and recall.

Table 3 summarizes the evaluation of our LLM. We achieved an accuracy of 82%. This accuracy can be perceived as low. This can be attributed to factors such as the size, diversity, and, especially, the lengths of the sentences of the dataset. The other models performed much lower, from 55% (CNN) to 75% (Multinomial Logistic Regression) and required the use of augmented datasets to achieve these results.

6 Validation by Subject Matter Experts

In this section, we describe the feedback of two subject matter experts (SME) on 1) the notion of impediments in Scrum projects; 2) the impediments classification that we proposed; and 3) the relevance of the development of a model to classify impediments that could be used in educational context and in the industry.

Table 3. Evaluation of the GPT-3.5 Turbo-based LLM to Classify Impediments

Metrics	Training	Testing
Accuracy	98	82
F1	98	82
Precision	98	86
Recall	98	83

6.1 Subject Matter Expert 1

Our first SME is a Scrum Alliance and SAFE agile coach with 10 years of industry experience. The SME is also a university instructor teaching software engineering and a trainer. The SME started by mentioning that it is a real differentiator on the job market when students are introduced to agile during their studies. The SME was asked to provide examples of impediments that arise in Scrum meetings. The cited examples were difficulties of working in multiple teams at the same time; unfinished user stories carried through sprints; project dependencies, i.e., relationships between tasks based on their sequence; and heterogeneous infrastructure. The SME agreed that the Scrum Guardian role is to help remove impediments, but, as the team matures, the team ends up removing its own impediments.

The first impressions of the SME on the impediments categories generated a discussion on the notion of impediments. The SME framed the notion of impediments in the answers to these two questions: “What is preventing the team to move forward and accomplish the sprint goal?” and “What is preventing the team to be a high performing team?” As Scrum is a timeboxed methodology, the SME did not think that “Time” would be a category of impediments in the industry. At the beginning of the sprint, teams need to plan based on the time to be allocated to the project. They should plan for 80% capacity. The SME also did not think that “Coding Skills” should be a category. With accountability being an important aspect of Scrum, teams know they have to learn the skills they do not have. The SME was going back and forth with the values and principles of agile and scrum methodologies, but also mentioned that no clear definition of impediments was available.

The SME was in favor of documenting Scrum meetings and using a model as described previously in the industry. Notes could be used by Scrum guardians, agile coaches, and management, and would be “useful to understand the mindset of the team and what help they need”. The SME thought that the proposed model was interesting for the class setting but would have liked more granularity with categories such as problems related to user stories, infrastructure and project dependencies. The SME also mentioned that an automated anti-pattern detection tool would be useful for agile coaches. Anti-patterns are behaviors that teams exhibit that would be detrimental in the long run [11]. It is recommended for teams to identify and remove such behaviors. They include absent product owner, micro-management, and weak definition of done.

6.2 Subject Matter Expert 2

Our second SME was a Scrum practitioner, author of several papers related to agile, and instructor who taught agile methodologies. The SME provided different examples of impediments cited during Scrum meetings from work experience: difficulty to distribute tasks to team members due to different levels of technical and business knowledge; problems of communication due to the presence of several point of contacts on the customer side; use of different languages in communications and documentation not understandable by everybody (multi-lingual teams); slow process to get infrastructure in place; and knowledge management issues. The SME mentioned that all the practices of Scrum are not implemented in the company. Scrums are not documented formally, the three Scrum questions are not always answered, and Scrums are often used to socialize and for team building. The Sprint Reviews are always documented. The SME insisted on the fact that most problems in software projects were due to communication and team dynamics rather than technical issues and that technical issues were easier to overcome.

The first impressions of the SME on the impediments categories that we identified were that they were very detailed and adapted to the educational setting for mobile app development specifically. The SME was interested in a generic model that would work for all types of projects, not focusing on technology choices. The SME believed that such a model would be very useful in the industry. Some of the categories could be combined such as “Android”, “Firebase” and “Coding Skills”. The SME would like to see categories related to “Testing”, “Business” and “Infrastructure” and would rename “Git” by “Supportive Tools” to englobe more of such a tool. The SME mentioned that the categories were adapted to the specific educational context described in this paper. Concerning the model, the SME thought that NLP can have an impact in detecting project issues on-the-fly.

7 Conclusion and Future Work

With this research, we sought to advance the understanding of impediments elicited in Scrum meetings. We identified classes of impediments in a capstone software engineering course where students developed mobile applications. We identified 9 categories of impediments in that context: Android, Coding Skills, Debugging, External Factors, Firebase/Database, Git/GitHub, Teamwork, Time Management, and UI/UX design. The categories are very specific to the type of project carried out in the course and capture the fact that students were beginners in Android, Git/GitHub, Firebase and Scrum. Such a classification is useful for instructors to understand the problems the teams encounter and for teams to improve their productivity. These findings also create a framework to guide students taking such courses; students know the difficulties they may encounter in their own projects.

We attempted to use NLP to analyze Scrum meeting notes and proposed a model that classifies teams’ impediments. The model is based on the GPT 3.5 Turbo LLM. It is an additional tool for instructors to detect the impediments met by students and address them in a timely manner and for the teams to better collaborate and build trust. The SME were convinced of the usefulness of such a tool in education to help the teams and support the instructors.

The proposed study, while focusing on education, can be of interest to the industry. SME imagined that the model could be used by Scrum Guardians, management, and Agile Coaches. They suggested categories of impediments that are more adapted to their industry and more generic, e.g., user stories, infrastructure, and project dependencies. They were also interested in more granularity in the categories.

We envision such a model for impediments classification to be used, live, during Scrum meetings and to generate notifications to be sent to a predefined set of project stakeholders in Slack or Discord communication tools. Such a model could also be integrated in Rally [20] or JIRA [22].

In the next steps of the study, we plan to use a larger dataset by integrating data from recent courses and tune the LLM to increase accuracy. We would like students to elaborate on their impediments and provide more explanation in the sentences collected in the dataset. We realized that the notion of impediments is understood differently by different practitioners. We plan to work on a more generic model for impediments classification for the industry using empirical data. We will also focus on other uses of NLP and computer vision in Scrum projects such as the detection of anti-patterns and summarization of Kanban Boards and Burndown Charts for Review sessions.

Acknowledgment. This work was approved under IRB 2023–06 and IRB 2023–90. We thank all the students and SMEs involved in this study.

References

1. Balog, M., Gaunt, A.L., Brockschmidt, M., Nowozin, S., Tarlow, D.: DeepCoder: Learning to write programs. In: International Conference on Representation Learning (ICLR) (2017)
2. Beecham, S., Noll, J., Clear, T.: Do we teach the right thing? A comparison of GSE education and practice. In: IEEE 12th International Conference on Global Software Engineering (ICGSE), Buenos Aires, Argentina, pp. 11–20 (2017)
3. Casillo, F., Deufemia, V., Gravino, C.: Detecting privacy requirements from user stories with NLP transfer learning models. *Inf. Softw. Technol.* **146**, 106853 (2022). <https://doi.org/10.1016/j.infsof.2022.106853>
4. Chang, H.-F., Shokrolah Shirazi, M.: Adapting scrum for software capstone courses. *Inform. Educ.* **21**(4), 605–634 (2022). <https://doi.org/10.15388/infedu.2022.25>
5. Chowdhary, K., Chowdhary, K.R.: Natural language processing. *Fundam. Artif. Intell.* 603–649 (2020). https://doi.org/10.1007/978-81-322-3972-7_19
6. Clear, T., Beecham, S.: Global software engineering education practice continuum special issue of the ACM transactions on computing education. *ACM Trans. Comput. Educ.* **19**(2), 1–8 (2019). <https://doi.org/10.1145/3294011>
7. Dada, O.A., Sanusi, I.T.: The adoption of software engineering practices in a scrum environment. *Afr. J. Sci. Technol. Innov. Dev.* **14**(6), 1429–1446 (2021). <https://doi.org/10.1080/20421338.2021.1955431>
8. Devlin, J., Chang, M.W., Lee, K.: BERT: Pretraining of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805 (2018)
9. Digital.AI. 16th State of Agile Report: resource center. (2022). <https://digital.ai/resource-center/analyst-reports/state-of-agile-report>
10. Elallaoui, M., Nafil, K., Touahno, R.: Automatic transformation of user stories into UML use case diagrams using NLP. *Techniques* (2018). <https://doi.org/10.1016/j.procs.2018.04.010>

11. Eloranta, V.-P., Koskimies, K., Mikkonen, T., Vuorinen, J.: Scrum anti-patterns an empirical study. In: 20th Asia-Pacific Software Engineering Conference (APSEC), Bangkok, Thailand, 2013, pp. 503–510 (2013). <https://doi.org/10.1109/APSEC.2013.72>
12. Feldt, R., de Oliveira, N., Francisco, G., Torkar, R.: Ways of applying artificial intelligence in software engineering. In: Proceedings of the 6th International Workshop on Realizing Artificial Intelligence Synergies in Software Engineering. Presented at the Gothenburg, Sweden, pp. 35–41 (2018). <https://doi.org/10.1145/3194104.3194109>
13. GitHub Copilot.: Your AI pair programmer (2022)
14. GPT-3.5 models. OpenAI. (2023). <https://platform.openai.com/docs/models/gpt-3-5>
15. Jiménez, O., Cliburn, D.: Scrum in the undergraduate computer science curriculum. *J. Comput. Sci. Coll.* **31**(4), 108–114 (2016)
16. Kondrashchenko, I., Kostromin, O.: Scikit-LLM: Sklearn meets large language models. (2023). <https://github.com/iryna-kondr/scikit-llm>
17. La, L., Guo, Q., Yang, D., Cao, Q.: Multiclass boosting with adaptive group-based kNN and its application in text categorization. *Math. Probl. Eng.* **2012**, 793490 (2012). <https://doi.org/10.1155/2012/793490>
18. Paasivaara, M., Vanhanen, J., Heikkilä, V.T., Lassenius, C., Itkonen, J., Laukkanen, E.: Do high and low performing student teams use scrum differently in capstone projects? In: IEEE/ACM 39th International Conference on Software Engineering: Software Engineering Education and Training Track (ICSE-SEET), Buenos Aires, Argentina, pp. 146–149 (2017). <https://doi.org/10.1109/ICSE-SEET.2017.22>
19. Pócsová, J., Bednářová, D., Bogdanovská, G., Mojžišová, A.: Implementation of Agile methodologies in an engineering course. *Educ. Sci.* **10**(11), 333 (2020). <https://doi.org/10.3390/educsci10110333>
20. Rally Software gives the business predictability and adaptability. <https://www.broadcom.com/products/software/value-stream-management/rally>
21. Rodriguez, G., Vidal, S., Marcos, C., Martinez Saucedo, A.C.: Evaluating students' perception of Scrum through a learning game. *Comput. Appl. Eng. Educ.* **30**(5), 1485–1497 (2022)
22. Sarkan, H.M., Ahmad, T.P.S., Bakar, A.A.: Using JIRA and Redmine in requirement development for agile methodology. In: Malaysian Conference in Software Engineering, Johor Bahru, Malaysia, pp. 408–413 (2011). <https://doi.org/10.1109/MySEC.2011.6140707>
23. Scharff, C., Verma, R.: Scrum to support mobile application development projects in a just-in-time learning context. In: Proceedings of the ICSE Workshop on Cooperative and Human Aspects of Software Engineering. Presented at the Cape Town, South Africa, pp. 25–31 (2010). <https://doi.org/10.1145/1833310.1833315>
24. Scharff, C., Heng, S., Kulkarni, V.: On the difficulties for students to adhere to Scrum on Global Software development projects: Preliminary results. In: Second International Workshop on Collaborative Teaching of Globally Distributed Software Development (CTGDSD), pp. 25–29. IEEE (2012). <https://doi.org/10.1109/CTGDSD.2012.6226946>
25. Sharma, S., Kumar, D.: Agile release planning using natural language processing algorithm. In: Amity International Conference on Artificial Intelligence (AICAI), Dubai, United Arab Emirates, pp. 934–938 (2019). <https://doi.org/10.1109/AICAI.2019.8701252>
26. Takeshi, K., Gu, S.S., Reid, M., Matsuo, Y., Iwasawa, Y.: Large language models are zero-shot reasoners. In: Advances in Neural Information Processing Systems, vol. 35, 22199–22213 (2022)
27. Tomas, P., Virginijus, M.: Application of logistic regression with part-of-the-speech tagging for multi-class text classification. In: IEEE 4th Workshop on Advances in Information, Electronic and Electrical Engineering (AIEEE), Vilnius, Lithuania, 2016, pp. 1–5 (2016). <https://doi.org/10.1109/AIEEE.2016.7821805>

28. Ulrich, S.: Training scrum with gamification: lessons learned after two teaching periods. In: IEEE Global Engineering Education Conference (EDUCON), Athens, Greece, pp. 754–761 (2017). <https://doi.org/10.1109/EDUCON.2017.7942932>
29. Verheyen, G.: Scrum a pocket guide. Van Haren Publishing (2019)
30. Wagh, R.: Using scrum for software engineering class projects. (2012). <https://doi.org/10.1109/AgileIndia.2012.17>
31. Yu, B., Deng, C., Bu, L.: Policy text classification algorithm based on BERT. In: 11th International Conference of Information and Communication Technology (ICTech)), Wuhan, China, 2022, pp. 488–491 (2022). <https://doi.org/10.1109/ICTech55460.2022.00103>
32. Zhang, X., Zhao, J., LeCun, Y.: Character-level convolutional networks for text classification. In: Advances in Neural Information Processing Systems, vol. 28, 1502–01710 (2015)